In [8]:
```
import numpy as np
import pandas as pd
```

In [9]:
```
data = {
    'Country': ['India', 'Japan', 'Brazil', 'Japan', 'Brazil',
                'India', 'Japan', 'Brazil', np.nan, 'India'],
    'Age': [25, 31, 28, 45, 39, 33, np.nan, 52, 41, 29],
    'Salary': [35000, 62000, 48000, 75000, np.nan,
               54000, 68000, 92000, 71000, 45000],
    'Purchased': ['Yes', 'No', 'Yes', 'No', 'Yes',
                  'No', 'Yes', 'No', 'Yes', 'No']
}

df = pd.DataFrame(data)
df
```

Out[9]:

| | Country | Age | Salary | Purchased |
|---|---|---|---|---|
| **0** | India | 25.0 | 35000.0 | Yes |
| **1** | Japan | 31.0 | 62000.0 | No |
| **2** | Brazil | 28.0 | 48000.0 | Yes |
| **3** | Japan | 45.0 | 75000.0 | No |
| **4** | Brazil | 39.0 | NaN | Yes |
| **5** | India | 33.0 | 54000.0 | No |
| **6** | Japan | NaN | 68000.0 | Yes |
| **7** | Brazil | 52.0 | 92000.0 | No |
| **8** | NaN | 41.0 | 71000.0 | Yes |
| **9** | India | 29.0 | 45000.0 | No |

In [10]:
```
df.head()
```

Out[10]:

| | Country | Age | Salary | Purchased |
|---|---|---|---|---|
| **0** | India | 25.0 | 35000.0 | Yes |
| **1** | Japan | 31.0 | 62000.0 | No |
| **2** | Brazil | 28.0 | 48000.0 | Yes |
| **3** | Japan | 45.0 | 75000.0 | No |
| **4** | Brazil | 39.0 | NaN | Yes |

In [11]:
```
df['Country'] = df['Country'].fillna(df['Country'].mode()[0])

df
```

Out[11]:

| | Country | Age | Salary | Purchased |
|---|---|---|---|---|
| **0** | India | 25.0 | 35000.0 | Yes |
| **1** | Japan | 31.0 | 62000.0 | No |
| **2** | Brazil | 28.0 | 48000.0 | Yes |
| **3** | Japan | 45.0 | 75000.0 | No |
| **4** | Brazil | 39.0 | NaN | Yes |
| **5** | India | 33.0 | 54000.0 | No |
| **6** | Japan | NaN | 68000.0 | Yes |
| **7** | Brazil | 52.0 | 92000.0 | No |
| **8** | Brazil | 41.0 | 71000.0 | Yes |
| **9** | India | 29.0 | 45000.0 | No |

In [12]:
```
features = df.iloc[:, :-1].values
features
```

Out[12]:
```
array([['India', 25.0, 35000.0],
       ['Japan', 31.0, 62000.0],
       ['Brazil', 28.0, 48000.0],
       ['Japan', 45.0, 75000.0],
       ['Brazil', 39.0, nan],
       ['India', 33.0, 54000.0],
       ['Japan', nan, 68000.0],
       ['Brazil', 52.0, 92000.0],
       ['Brazil', 41.0, 71000.0],
       ['India', 29.0, 45000.0]], dtype=object)
```

In [13]:
```
label = df.iloc[:, -1].values
label
```

Out[13]:
```
<ArrowStringArray>
['Yes', 'No', 'Yes', 'No', 'Yes', 'No', 'Yes', 'No', 'Yes', 'No']
Length: 10, dtype: str
```

In [14]:
```
from sklearn.impute import SimpleImputer
```

In [15]:
```
age = SimpleImputer(strategy="mean", missing_values=np.nan)
Salary = SimpleImputer(strategy="mean", missing_values=np.nan)
```

In [16]:
```
age.fit(features[:, [1]])
```

Out[16]:

▾ SimpleImputer

▸ Parameters

▸ Fitted attributes

▸ 1 feature

In [17]:
```
Salary.fit(features[:, [2]])
```

Out[17]:

▾ SimpleImputer

▸ Parameters

▸ Fitted attributes

▸ 1 feature

In [18]:
```
features[:, [1]] = age.transform(features[:, [1]])
features[:, [2]] = Salary.transform(features[:, [2]])

features
```

Out[18]:
```
array([['India', 25.0, 35000.0],
       ['Japan', 31.0, 62000.0],
       ['Brazil', 28.0, 48000.0],
       ['Japan', 45.0, 75000.0],
       ['Brazil', 39.0, 61111.11111111111],
       ['India', 33.0, 54000.0],
       ['Japan', 35.888888888888886, 68000.0],
       ['Brazil', 52.0, 92000.0],
       ['Brazil', 41.0, 71000.0],
       ['India', 29.0, 45000.0]], dtype=object)
```

In [19]:
```
from sklearn.preprocessing import OneHotEncoder

oh = OneHotEncoder(sparse_output=False)
Country = oh.fit_transform(features[:, [0]])

Country
```

Out[19]:
```
array([[0., 1., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [1., 0., 0.],
       [0., 1., 0.]])
```

In [20]:
```
final_set = np.concatenate((Country, features[:, [1, 2]]), axis=1)

final_set
```

Out[20]:
```
array([[0.0, 1.0, 0.0, 25.0, 35000.0],
       [0.0, 0.0, 1.0, 31.0, 62000.0],
       [1.0, 0.0, 0.0, 28.0, 48000.0],
       [0.0, 0.0, 1.0, 45.0, 75000.0],
       [1.0, 0.0, 0.0, 39.0, 61111.11111111111],
       [0.0, 1.0, 0.0, 33.0, 54000.0],
       [0.0, 0.0, 1.0, 35.888888888888886, 68000.0],
       [1.0, 0.0, 0.0, 52.0, 92000.0],
       [1.0, 0.0, 0.0, 41.0, 71000.0],
       [0.0, 1.0, 0.0, 29.0, 45000.0]], dtype=object)
```

In [21]:
```
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
sc.fit(final_set)

feat_standard_scaler = sc.transform(final_set)

feat_standard_scaler
```

Out[21]:
```
array([[-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
        -1.36229417e+00, -1.66583735e+00],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
        -6.11642281e-01,  5.67093565e-02],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
        -9.86968227e-01, -8.36463009e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
         1.13987880e+00,  8.86083696e-01],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
         3.89226906e-01,  0.00000000e+00],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
        -3.61424984e-01, -4.53674852e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
        -8.88950413e-16,  4.39497513e-01],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
         2.01563934e+00,  1.97065014e+00],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
         6.39444203e-01,  6.30891591e-01],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
        -8.61859578e-01, -1.02785709e+00]])
```

In [22]:
```
from sklearn.preprocessing import MinMaxScaler

mms = MinMaxScaler(feature_range=(0, 1))
mms.fit(final_set)

feat_minmax_scaler = mms.transform(final_set)

feat_minmax_scaler
```

Out[22]:
```
array([[0.        , 1.        , 0.        , 0.        , 0.        ],
       [0.        , 0.        , 1.        , 0.22222222, 0.47368421],
       [1.        , 0.        , 0.        , 0.11111111, 0.22807018],
       [0.        , 0.        , 1.        , 0.74074074, 0.70175439],
       [1.        , 0.        , 0.        , 0.51851852, 0.45808967],
       [0.        , 1.        , 0.        , 0.2962963 , 0.33333333],
       [0.        , 0.        , 1.        , 0.40329218, 0.57894737],
       [1.        , 0.        , 0.        , 1.        , 1.        ],
       [1.        , 0.        , 0.        , 0.59259259, 0.63157895],
       [0.        , 1.        , 0.        , 0.14814815, 0.1754386 ]])
```

In [ ]: